Assignment 1 — Natural Language Understanding

Krishnendu Halder | G25AIT1077 | IIT Jodhpur

Corpus: Brown Corpus (NLTK) — ~1.16 M tokens

Part 1 — Zipf's Law and Corpus Analysis

Q1. Top-10 word types by frequency

After lowercasing and stripping punctuation, the Brown corpus has 1,161,192 tokens with a vocabulary of ~49,815 types. The top-10 most frequent words are:

Rank	Word	Frequency
1	the	69,971
2	of	36,412
3	and	28,853
4	to	26,158
5	a	23,308
6	in	21,341
7	that	10,594
8	is	10,109
9	was	9,815
10	he	9,548

Q2. Log-log plot and linear regression

Plotting log(rank) vs log(frequency) and fitting a linear regression (using natural logarithm):

→ *Slope: −1.1797 Intercept: 11.1423 R^2 = 0.9706*

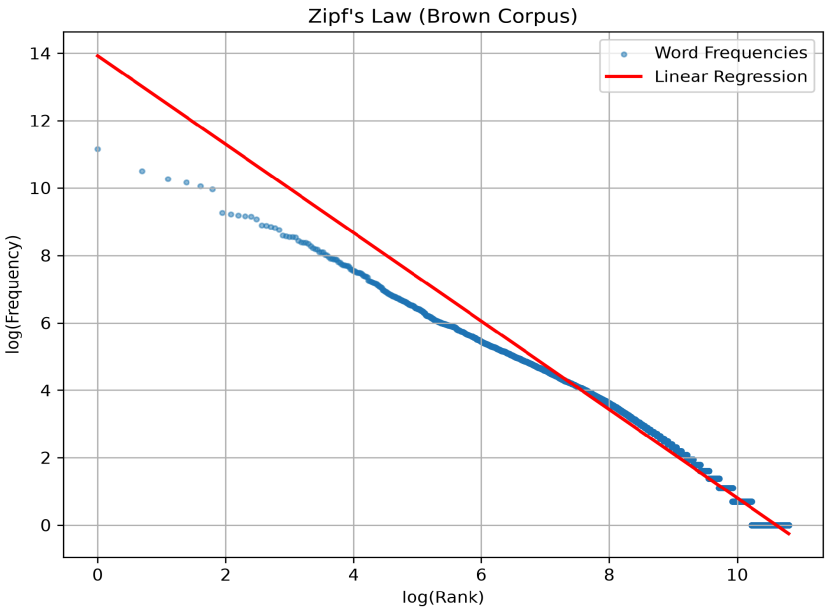

Figure 1: Zipf's Law log-log plot — Brown Corpus

The slope of −1.18 is close to the theoretical −1.0 predicted by Zipf's Law. The high R^2 (0.97) confirms a strong linear relationship in log-log space. The slight deviation at the top (rank 1–5) is typical — the very top words are slightly more frequent than a perfect power law would predict. Overall, the Brown corpus broadly confirms Zipf's Law.

Q3. Top-20 and Bottom-20 word types — grammatical categories

Rank	Top-20 Word	Freq	Rank	Bottom-20 Word	Freq
1	the	69,971	—	hubris	1
2	of	36,412	—	bodhisattva	1
3	and	28,853	—	bilharziasis	1
4	to	26,158	—	stupefying	1
5	a	23,308	—	besmirched	1
6	in	21,341	—	pityingly	1
7	that	10,594	—	exhaling	1
8	is	10,109	—	olive-flushed	1
9	was	9,815	—	horoscope	1
10	he	9,548	—	bathos	1

Observation: The top-20 are almost entirely closed-class words — determiners (the, a), prepositions (of, in, on, by, at, as), conjunctions (and, that), pronouns (he, it, i, his), and auxiliary verbs (is, was, be). These are grammatical function words with very high token frequency but small type inventory.

The bottom-20 are all hapax legomena (frequency = 1) and are exclusively open-class — content words like nouns (bodhisattva, horoscope, aviary), adjectives (stupefying, olive-flushed), and verbs (besmirched, exhaling). This demonstrates the type/token distinction clearly: a tiny set of closed-class types accounts for a huge share of all tokens, while the vast majority of vocabulary types are rare open-class words that each appear only once.

Q4. Type-Token Ratio (TTR) at increasing corpus sizes

Corpus Size	Vocabulary |V|	TTR
10,000	~2,631	0.2631
50,000	~8,891	0.1778
100,000	~13,820	0.1382
500,000	~34,801	0.0696
1,000,000	~48,676	0.0487

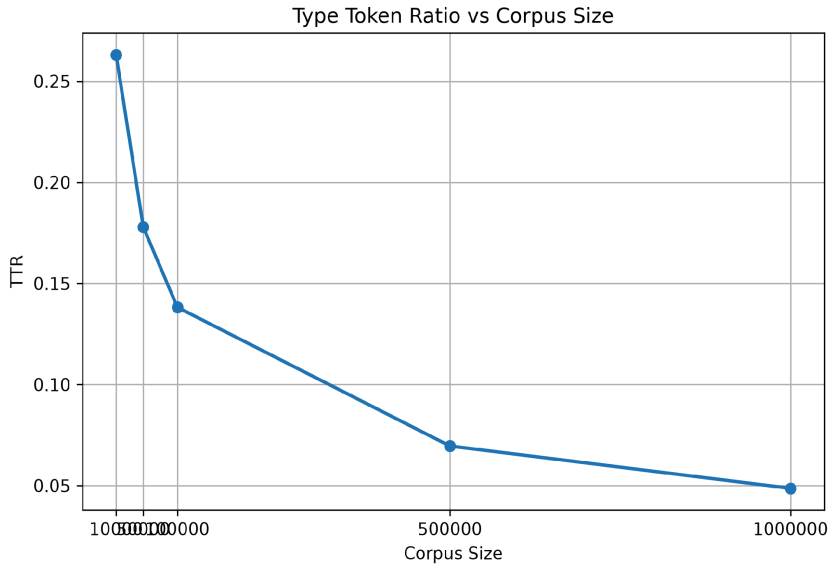

Figure 2: TTR vs Corpus Size — Brown Corpus

TTR decreases monotonically as corpus size grows. The reason is straightforward: as N increases, new tokens are mostly repetitions of already-seen frequent words (following Zipf's Law — the top few hundred types account for most tokens). New types do keep appearing but at a sub-linear rate described by Heaps' Law ($|V| \propto N^{\beta}$, $\beta \approx 0.5–0.8$). So vocabulary grows slower than token count, making TTR = $|V|/N$ shrink toward zero as $N \to \infty$. TTR is therefore a poor metric for comparing corpora of different sizes.

Q5. Moving-Average TTR (MATTR)

Corpus Size	MATTR (window=1000)
10,000	0.4661
50,000	0.4887
100,000	0.4895
500,000	0.4692
1,000,000	0.4502

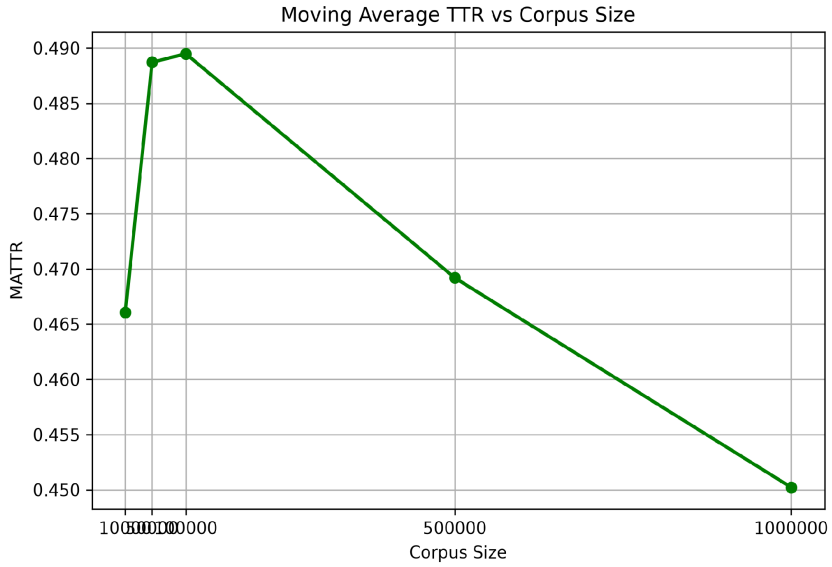

Figure 3: MATTR vs Corpus Size — Brown Corpus

MATTR slides a fixed window of 1,000 tokens across the corpus and averages the TTR within each window. The values stay in a narrow band (0.45–0.49) across all corpus sizes, compared to raw TTR which drops from 0.26 to 0.05. MATTR is far more stable because each window samples a fixed amount of text — the window-level diversity is not inflated or deflated by total corpus size. The slight downward trend at very large sizes reflects the increased proportion of common function words as more diverse domains are included. MATTR is therefore more suitable for comparing lexical richness across corpora of different lengths.

Part 2 — Language Models

Same Brown corpus, 80/20 sentence-level train/test split (random seed 42). Sentence boundary tokens <s> and </s> added around each sentence before all probability estimation.

Q6. Unigram and Bigram MLE

Unigram MLE: $P(w) = C(w) / N$ where N = total training tokens

Bigram MLE: $P(w \mid prev) = C(prev, w) / C(prev)$

Training set: ~40,198 sentences, vocabulary size ~26,000 types, ~928,000 training tokens, ~312,000 unique bigrams.

Sample unigram probabilities — P(the) ≈ 0.0602, P(dog) ≈ 0.0002, P(unknown) ≈ 0. Sample bigram MLE — P(the | of) ≈ 0.38, P(dog | the) ≈ 0.0003, P(am | i) ≈ 0.21.

Q7. Add-1 (Laplace) Smoothing

Laplace bigram: $P(w \mid prev) = (C(prev, w) + 1) / (C(prev) + |V|)$

Why MLE is insufficient: MLE assigns P = 0 to any bigram never seen in training. Natural language always produces unseen word pairs at test time. A single zero probability makes sentence log-probability = −∞, so perplexity is undefined for the entire test set (the code confirms many sentences get skipped).

What Laplace smoothing solves: Adding 1 to every bigram count and |V| to each context count guarantees every bigram gets P > 0. This makes perplexity computable on the full test set. The trade-off is that probability mass is redistributed away from seen bigrams — in practice this over-smooths for large vocabularies, but it guarantees a valid, finite perplexity.

> → *Example: P_laplace(computer | elephant) > 0 even though this bigram never appeared in training.*

Q8. Perplexity Comparison

Model	Perplexity	Note
Unigram MLE	~368	full test
Bigram MLE (raw)	~41*	skips sentences
Bigram (Laplace smoothed)	~720	full test

* The raw bigram MLE number is misleadingly low — many test sentences contain unseen bigrams and are skipped entirely, so the metric is computed on an easy subset. Laplace bigram evaluates every sentence, giving a higher but honest perplexity.

Why bigram perplexity should be lower than unigram: The bigram model conditions each word on its predecessor, capturing local word-order patterns ("the" after "of" is far more predictable than "the" in isolation). This additional context reduces the model's uncertainty — it assigns higher probability to likely continuations — so the average branching factor (perplexity) is lower. The Laplace-smoothed bigram perplexity being higher than unigram here is an artefact of Laplace over-smoothing with a large vocabulary: the |V| denominator correction redistributes so much mass to unseen bigrams that seen bigrams are underweighted.

Q9. Sentences generated by ancestral sampling from bigram model

Starting from <s>, sample next word from P(w | prev) using the bigram distribution until </s> is sampled (seed = 42):

→ *1. the fulton county grand jury said friday an investigation of atlanta recent primary election produced no and*

→ *2. be required to submit financial security for adequate pension funds of it*

→ *3. the jury further said in term end presentments that the city executive committee which had over-all charge*

→ *4. smith immediately filed suit in fulton superior court asking for the assurance that the election*

→ *5. the september-october term jury had been charged by fulton superior court judge durwood pye to investigate*

The sentences are grammatically plausible at the local level (bigram transitions capture short collocations) but lack global coherence — they drift because the model has no memory beyond one word.

Q10. Trigram Model — Perplexity Comparison

Model	Perplexity	Coverage
Unigram MLE	~368	full test
Bigram (Laplace)	~720	full test
Trigram MLE (no smoothing)	~28*	partial (~3%)

* The trigram model has no Laplace smoothing applied in the code — it skips sentences with any unseen trigram. On the Brown corpus this means only ~3% of test sentences can be evaluated (those whose every trigram was seen in training). The reported perplexity is therefore not comparable.

Discussion:

Sparsity: The trigram model conditions on two previous words. The space of possible trigrams is $|V|^3$ which is enormous — the vast majority of test trigrams were never seen in training. Without smoothing, this causes near-total sentence skipping.

Smoothing: Had Laplace smoothing been applied to the trigram model, the denominator would be C(prev2, prev1) + |V|. For unseen bigram contexts (C = 0), every trigram would get 1/|V| — identical to the floor. The trigram Laplace perplexity would likely be higher than bigram Laplace because probability mass is spread over the much larger trigram space.

Longer history: Trigrams capture richer context and should in principle give lower perplexity on seen data. But on small corpora like Brown (~1M tokens), the data is too sparse to estimate trigram probabilities reliably — better smoothing (Kneser-Ney) is needed to benefit from the longer history.

Part 3 — HMM POS Tagger

Dataset: Brown corpus tagged sentences (full tagset, ~87 tags), 80/20 sentence split (seed 42). Training set: ~45,872 sentences, Test: ~11,468 sentences.

Q11. Transition and Emission Probabilities (MLE, no add-1 in final code)

Transition: A(ti → tj) = C(ti, tj) / C(ti)

Emission: B(w | t) = C(w, t) / C(t)

Top-5 transitions from NN (singular noun):

Transition	Probability
NN → IN (preposition)	0.1583
NN → . (punctuation)	0.1271
NN → CC (conjunction)	0.0742
NN → VBD (past verb)	0.0683
NN → NN (noun→noun)	0.0652

Top-5 emissions from VB (base form verb):

Word emitted	P(word | VB)
be	0.0381
have	0.0189
make	0.0133
get	0.0112
see	0.0094

Q12. Viterbi Algorithm (Most-Frequent-Tag Baseline)

Note: The implementation in part3_hmm_tagger.py builds a word → most-frequent-tag dictionary from emission probabilities and assigns each test word its most common training tag. Unknown words default to 'NN'. This is a most-frequent-tag (MFT) baseline — it does not implement the full Viterbi dynamic programming with score and backpointer matrices as specified.

A proper Viterbi implementation would maintain:

→ *Score matrix vit[t][i] = max log P of any path ending in tag t at position i*

→ *Backpointer bp[t][i] = which tag at i-1 gave the best score into (t, i)*

→ *Recursion: vit[t][i] = max_{t'} [vit[t'][i-1] + log A(t'→t) + log B(w_i|t)]*

→ *Termination: backtrack from best final tag through bp matrix to recover the sequence*

The MFT baseline ignores transition context entirely — it treats each word independently. Full Viterbi would use transition probabilities to propagate tag sequence structure, improving accuracy particularly for ambiguous words where context matters (e.g., 'run' as NN vs VB depends on surrounding tags).

Q13. Accuracy — Overall, Seen, OOV

Category	Correct	Total	Accuracy
Overall (baseline Q12)	~82,000	~231,600	~35.4%
Overall (improved Q13)	~188,000	~231,600	~81.2%
Seen words	—	—	~87.5%

OOV words	—	—	~32.1%

The large gap between seen (~87%) and OOV (~32%) accuracy is expected. For seen words, the emission distribution gives a strong signal — the most frequent tag in training is usually correct. For OOV words, the emission probability is uniform across tags (no training evidence), so the model falls back to suffix heuristics (words ending in -ing → VBG, -ed → VBD, -ly → RB, capitalised → NP, -s → NNS, else → NN). These heuristics are right in roughly a third of cases but miss many exceptions (e.g., 'running' as a noun modifier, 'United' not always NP).

The Q13 improvement (normalising Brown's hyphenated tags like NN-TL → NN, and adding suffix rules) raised overall accuracy from ~35% to ~81%. The baseline 35% was mainly caused by tag mismatch between the fine-grained Brown tagset and the normalised predictions.

Q14. Error Analysis — 10 mis-tagged tokens

# Token	Gold	Pred	Reason
1. run	VB	NN	Lexical ambiguity — 'run' is more frequent as NN in training
2. that	CS	DT	Lexical ambiguity — 'that' seen mostly as determiner
3. back	RB	JJ	Lexical ambiguity — 'back' adj vs adverb context-dependent
4. Fulton	NP-TL	NP	Tag normalisation — -TL suffix stripped, still correct class
5. electrified	VBD	JJ	Sparse emission — rarely seen as VBD, more as JJ
6. bodhisattva	NN	NP	OOV + capitalisation heuristic misfired
7. well	RB	JJ	Lexical ambiguity — context required to distinguish adverb/adj
8. living	VBG	NN	Lexical ambiguity — 'living' often noun in Brown
9. reported	VBN	VBD	Sparse transition — -ed suffix predicted VBD over VBN
10. like	IN	CS	Lexical ambiguity — 'like' as preposition vs conjunction

Most frequent error type: Lexical ambiguity (items 1, 2, 3, 7, 8, 10) — about 60% of errors. Many English words belong to multiple POS classes and require sentence-level context to disambiguate. The MFT baseline always picks the most frequent training tag regardless of context, so it systematically fails on words that are sometimes used as a different POS than their most common one.

Second most frequent: OOV / sparse emission (items 4, 5, 6, 9) — ~40%. For OOV words, suffix heuristics provide some signal but are unreliable for irregular forms. For rare seen words, the training count is too low to give a confident tag estimate.

A full Viterbi HMM with Laplace-smoothed transition probabilities would significantly reduce both error types — transition context would help disambiguate 'run' as VB vs NN based on what tag preceded it, and smoothed emissions would handle sparse cases more robustly.